

## A2UI Specifications — Reference

### # A2UI Specifications — Reference

> **Source:** [a2ui.org](https://a2ui.org/) — copied verbatim into this repository on **2026-07-27** for offline reference.  
> **Status:** Living documents. The upstream specification is automatically included from `specification/v0_9_1/docs/`; any upstream updates appear there first. This file is a snapshot.  
> **Current production release:** **v0.9.1**. For the legacy version, see v0.8 (Legacy) upstream.

#### **Contents:**

- §1 — A2UI Protocol v0.9.1** (`a2ui_protocol.md`)
- §2 — A2UI Basic Catalog Implementation Guide v0.9.1** (`basic_catalog_implementation_guide.md`)
- §3 — A2UI Extension for A2A Protocol v0.9.1** (`a2ui_extension_specification.md`)

Related upstream documentation: v0.9 Protocol Specification (previous stable), Evolution Guide: v0.9 → v0.9.1.

---  
---

### # §1 — A2UI Protocol v0.9.1

**A Specification for a JSON-Based, Streaming UI Protocol.**

**Version:** 0.9.1 **Status:** Current Production **Created:** Nov 20, 2025 **Last Updated:** Dec 3, 2025

#### ## Introduction

The A2UI Protocol is designed for dynamically rendering user interfaces from a stream of JSON objects sent from a server (Agent). Its core philosophy emphasizes a clean separation of UI structure and application data, enabling progressive rendering as the client processes each message.

Communication occurs via a stream of JSON objects. The client parses each object as a distinct message and incrementally builds or updates the UI. The server-to-client protocol defines four message types:

- **createSurface**: Signals the client to create a new surface and begin rendering it.
- **updateComponents**: Provides a list of component definitions to be added to or updated in a specific surface.

- **updateDataModel**: Provides new data to be inserted into or to replace a surface's data model.
- **deleteSurface**: Explicitly removes a surface and its contents from the UI.

## ## Changes from previous versions

Version 0.9 introduced A2UI's prompt-first protocol family. While v0.8 was optimized for LLMs that support structured output, v0.9 is designed to be embedded directly within a model's prompt. The LLM is then asked to produce JSON that matches the provided examples and schema descriptions.

This "prompt-first" approach offers several advantages:

- **Richer schema**: The protocol is no longer limited by the constraints of structured output formats. This allows for more readable, complex, and expressive component catalogs.
- **Modularity**: The schema is now refactored into separate, more manageable components (e.g., `common_types.json`, `catalogs/basic/catalog.json`, `server_to_client.json`), improving maintainability and modularity.

The main disadvantage of this approach is that it requires more complex post-generation validation, as the LLM is not strictly constrained by the schema. This requires robust error handling and correction, so the system can identify discrepancies and attempt to fix them before rendering, or request a retry or correction from the LLM.

See the evolution guide for a detailed explanation of the differences between v0.9 and v0.9.1.

## ## Protocol overview & data flow

The A2UI protocol uses a unidirectional stream of JSON messages from the server to the client to describe and update the UI. The client consumes this stream, builds the UI, and renders it. User interactions are handled separately, typically by sending events to a different endpoint, which may in turn trigger new messages on the UI stream.

Here is an example sequence of events (which don't have to be in exactly this order):

1. **Create Surface**: The server sends a `createSurface` message to initialize the surface.
2. **Update Surface**: Once a surface has been created, the server sends one or more `updateComponents` messages containing the definitions for all the components that will be part of the surface.
3. **Update Data Model**: Once a surface has been created, the server can send `updateDataModel` messages at any time to populate or change the data that the UI components will display.
4. **Render**: The client renders the UI for the surface, using the component definitions to build the structure and the data model to populate the content.

5. **\*\*Dynamic updates\*\***: As the user interacts with the application or as new information becomes available, the server can send additional `updateComponents` and `updateDataModel` messages to dynamically change the UI.
6. **\*\*Delete Surface\*\***: When a UI region is no longer needed, the server sends a `deleteSurface` message to remove it.

...

| User | Client | Server                                                                            |
|------|--------|-----------------------------------------------------------------------------------|
|      |        |                                                                                   |
|      |        | Time passes, user interacts, or new data arrives...                               |
|      |        | Client re-renders the UI to reflect changes                                       |
|      |        | Client removes the UI for the "main" surface                                      |
|      |        |                                                                                   |
|      |        | 1. <code>createSurface(surfaceId: "main")</code>                                  |
|      |        | 2. <code>updateComponents(surfaceId: "main", components: [...])</code>            |
|      |        | 3. <code>updateDataModel(surfaceId: "main", path: "/user", value: "Alice")</code> |
|      |        | Interact with UI (e.g. click button)                                              |
|      |        | <code>action(name: "submit", context: {...})</code>                               |
|      |        | (UI is displayed)                                                                 |
|      |        | 4. <code>updateComponents</code> or <code>updateDataModel</code> (Dynamic Update) |
|      |        | (UI is updated)                                                                   |
|      |        | 5. <code>deleteSurface(surfaceId: "main")</code>                                  |
|      |        | (UI is gone)                                                                      |

...

## ## Transport decoupling

The A2UI protocol is designed to be transport-agnostic. It defines the JSON message structure and the semantic contract between the server (Agent) and the client (Renderer), but it does not mandate a specific transport layer.

### ### The transport contract

To support A2UI, a transport layer must fulfill the following contract:

- **\*\*Reliable delivery\*\***: Messages must be delivered in the order they were generated. A2UI relies on stateful updates (e.g., creating a surface before updating it), so out-of-order delivery can corrupt the UI state.
- **\*\*Message framing\*\***: The transport must clearly delimit individual JSON envelope messages (e.g., using newlines in JSONL, WebSocket frames, or SSE events).
- **\*\*Metadata support\*\***: The transport must provide a mechanism to associate metadata with messages. This is critical for:
  - **\*\*Data model synchronization\*\***: The `sendDataModel` feature requires the client to send the current data model state as metadata alongside user actions.

- **Capabilities exchange**: Client capabilities (supported catalogs, custom components) and Server capabilities are exchanged via metadata or transport-specific handshakes (like Agent Cards in A2A or initialization in MCP).
- **Bidirectional capability (optional)**: While the rendering stream is unidirectional (Server -> Client), interactive applications require a return channel for action messages (Client -> Server).

### ### Transport bindings

While A2UI is agnostic, it is most commonly used with the following transports.

**A2A (Agent2Agent) binding** — A2A is an excellent transport option for A2UI in agentic systems, extending A2A with additional payloads. A2A is uniquely capable of handling remote agent communication, and can also provide a secure and efficient transport between an agentic backend and front end application.

- **Message mapping**: Each A2UI envelope (e.g., updateComponents) corresponds to the payload of a single A2A message Part.
- **Metadata**:
  - **Data model**: When sendDataModel is active, the client's a2uiClientDataModel object is placed in the metadata field of the A2A message.
  - **Capabilities**: The a2uiClientCapabilities object is placed in the metadata field of every A2A Message sent from the client to the server.
  - **Context**: A2UI sessions typically map to A2A contextId. All messages for a set of related surfaces should share the same contextId.

**AG UI (Agent to User Interface) binding** — AG-UI is also an excellent transport option for A2UI Agent–User Interaction protocol. AG UI provides convenient integrations into many agent frameworks and frontends. AG UI provides low latency and shared state message passing between front ends and agentic backends.

**Other transports** — A2UI can also be carried over:

- **MCP (Model Context Protocol)**: Delivered as tool outputs or resource subscriptions.
- **SSE with JSON RPC**: Standard server-sent events for web integrations that support streaming, and JSON RPC for client-server communication.
- **WebSockets**: For bidirectional, real-time sessions.
- **REST**: For simple use case, REST APIs will work but lack streaming capabilities.

### ## The protocol schemas

A2UI v0.9.1 is defined by three interacting JSON schemas.

### ### Common types

The `common\_types.json` schema defines reusable primitives used throughout the protocol.

- **DynamicString / DynamicNumber / DynamicBoolean / DynamicStringList**: The core of the data binding system. Any property that can be bound to data is defined as a **Dynamic\*** type. It accepts either a literal value, a path string (JSON Pointer), or a FunctionCall (function call).
- **ChildList**: Defines how containers hold children. It supports:
  - **array**: A static array of ComponentId component references.
  - **object**: A template for generating children from a data binding list (requires a template componentId and a data binding path).
- **ComponentId**: A reference to the unique ID of another component within the same surface.

### Server to client message structure: the envelope

The ``server_to_client.json`` schema is the top-level entry point. Every message streamed by the server must validate against this schema. It handles the message dispatching.

### The Basic Catalog

The ``catalogs/basic/catalog.json`` schema contains the definitions for all specific UI components (e.g., Text, Button, Row), functions (e.g., required, email), and the theme schema.

**Swappable Catalogs & Validation**

The ``server_to_client.json`` envelope schema is designed to be catalog-agnostic. It references components and themes using a placeholder filename: ``catalog.json`` (specifically ``$ref: "catalog.json#/$defs/anyComponent"`` and ``$ref: "catalog.json#/$defs/theme"``).

To validate A2UI messages:

- **Basic Catalog**: Map ``catalog.json`` to ``catalogs/basic/catalog.json``.
- **Client Catalog**: Map ``catalog.json`` to your own catalog file (e.g., ``my_company_catalog.json``).

This indirection allows the same core envelope schema to be used with any compliant component catalog without modification.

Defining your own catalog allows you to restrict the agent to using exactly the components and visual language that exist in your application. To use your own catalog, simply include it in the prompt in place of the basic catalog. It should have the same form as the basic catalog and use common elements in the ``common_types.json`` schema.

### Validator compliance when defining catalogs

To ensure that automated validators can verify the integrity of your UI tree (checking that parents reference existing children), any catalog you define **MUST** adhere to the following strict typing rules:

- **\*\*Single child references\*\***: Any property that holds the ID of another component **MUST** use the `ComponentId` type defined in `common_types.json`.
  - Use: ``"$ref": "common_types.json#/$defs/ComponentId"``
  - Do NOT use: ``"type": "string"``
- **\*\*List references\*\***: Any property that holds a list of children or a template **MUST** use the `ChildList` type.
  - Use: ``"$ref": "common_types.json#/$defs/ChildList"``

Validators determine which fields represent structural links by looking for these specific schema references. If you use a raw string type for an ID, the validator will treat it as static text (like a URL or label) and will not check if the target component exists.

## ## Envelope message structure

The envelope defines four primary message types, and every message streamed by the server must be a JSON object containing exactly one of the following keys: ``createSurface``, ``updateComponents``, ``updateDataModel``, or ``deleteSurface``. The key indicates the type of message, and these are the messages that make up each message in the protocol stream.

### ### createSurface

This message signals the client to create a new surface and begin rendering it. A surface must be created before any `updateComponents` or `updateDataModel` messages can be sent to it. While typically achieved by the agent sending a `createSurface` message, an agent may skip this if it knows the surface has already been created (e.g., by another agent). Once a surface is created, its `surfaceId` and `catalogId` are fixed; to reconfigure them, the surface must be deleted and recreated. It is an error to send `createSurface` for a `surfaceId` that already exists without first deleting it. One of the components in one of the component lists **MUST** have an id of ``root`` to serve as the root of the component tree.

### **\*\*Properties:\*\***

- **\*\*surfaceId\*\*** (string, required): The unique identifier for the UI surface to be rendered.
- **\*\*catalogId\*\*** (string, required): A string that uniquely identifies the catalog (components and functions) used for this surface. Note that `catalogId` is a string identifier, not a resolvable URI; while it is conventionally formatted as a URI (e.g., ``https://mycompany.com/1.0/somecatalog``) to avoid naming collisions across organizations, it does not need to point to any deployed resource or downloadable file. Client and server developers must agree on shared catalogs with well-known IDs in order to build systems that are compatible with each other.
- **\*\*theme\*\*** (object, optional): A JSON object containing theme parameters (e.g., `primaryColor`) defined in the catalog's theme schema.
- **\*\*sendDataModel\*\*** (boolean, optional): If true, the client will send the full data model of this surface in the metadata of every message sent to the server (via the Transport's metadata mechanism). This ensures the surface owner receives the full current state of the UI alongside the user's action or query. Defaults to false.

**\*\*Example:\*\***

```
```json
{
  "version": "v0.9.1",
  "createSurface": {
    "surfaceId": "user_profile_card",
    "catalogId": "https://a2ui.org/specification/v0_9_1/catalogs/basic/catalog.json",
    "theme": {
      "primaryColor": "#00BFFF"
    },
    "sendDataModel": true
  }
}
```
```

### ### updateComponents

This message provides a list of UI components to be added to or updated within a specific surface. The components are provided as a flat list, and their relationships are defined by ID references in an adjacency list. This message may only be sent to a surface that has already been created. Note that components may reference children or data bindings that do not yet exist; clients should handle this gracefully by rendering placeholders (progressive rendering).

**\*\*Properties:\*\***

- **\*\*surfaceId\*\*** (string, required): The unique identifier for the UI surface to be updated. This is typically a name with meaning (e.g. "user\_profile\_card"), and it has to be unique within the context of the GenUI session.
- **\*\*components\*\*** (array, required): A list of component objects. The components are provided as a flat list, and their relationships are defined by ID references in an adjacency list.

**\*\*Example:\*\***

```
```json
{
  "version": "v0.9.1",
  "updateComponents": {
    "surfaceId": "user_profile_card",
    "components": [
      {
        "id": "root",
        "component": "Column",
        "children": ["user_name", "user_title"]
      },
      {

```

```

        "id": "user_name",
        "component": "Text",
        "text": "John Doe"
    },
    {
        "id": "user_title",
        "component": "Text",
        "text": "Software Engineer"
    }
]
}
}
...

```

### ### updateDataModel

This message is used to send or update the data that populates the UI components. It allows the server to change the UI's content without resending the entire component structure. The updateDataModel message replaces the value at the specified path with the new content. If path is omitted (or is `^`), the entire data model for the surface is replaced.

#### **\*\*Properties:\*\***

- **\*\*surfaceId\*\*** (string, required): The unique identifier for the UI surface this data model update applies to.
- **\*\*path\*\*** (string, optional): A JSON Pointer to the location in the data model to update. Defaults to `^`.
- **\*\*value\*\*** (any, optional): The new value for the specified path. If omitted, the key at path is removed.

#### **\*\*Example:\*\***

```

```json
{
  "version": "v0.9.1",
  "updateDataModel": {
    "surfaceId": "user_profile_card",
    "path": "/user/name",
    "value": "Jane Doe"
  }
}
```

```

### ### deleteSurface



This message instructs the client to remove a surface and all its associated components and data from the UI.

**\*\*Properties:\*\***

- **\*\*surfaceId\*\*** (string, required): The unique identifier for the UI surface to be deleted.

**\*\*Example:\*\***

```
```json
{
  "version": "v0.9.1",
  "deleteSurface": {
    "surfaceId": "user_profile_card"
  }
}
```
```

### ### Example Stream

The following example demonstrates a complete interaction to render a Contact Form, expressed as a JSONL stream.

```
```jsonl
{"version": "v0.9.1", "createSurface": {"surfaceId": "contact_form_1", "catalogId": "https://a2ui.org/specification/v0_9_1/catalogs/basic/catalog.json"}}
{"version": "v0.9.1", "updateComponents": {"surfaceId": "contact_form_1", "components": [
  {"id": "root", "component": "Card", "child": "form_container"},
  {"id": "form_container", "component": "Column", "children": [
    "header_row", "name_row", "email_group", "phone_group", "pref_group", "divider_1", "newsletter_checkbox", "submit_button"], "justify": "start", "align": "stretch"},
  {"id": "header_row", "component": "Row", "children": [
    "header_icon", "header_text"], "align": "center"},
  {"id": "header_icon", "component": "Icon", "name": "mail"},
  {"id": "header_text", "component": "Text", "text": "# Contact Us", "variant": "h2"},
  {"id": "name_row", "component": "Row", "children": [
    "first_name_group", "last_name_group"], "justify": "spaceBetween"},
  {"id": "first_name_group", "component": "Column", "children": [
    "first_name_label", "first_name_field"], "weight": 1},
  {"id": "first_name_label", "component": "Text", "text": "First Name", "variant": "caption"},
  {"id": "first_name_field", "component": "TextField", "label": "First Name", "value": {"path": "/contact/firstName"}, "variant": "shortText"},
  {"id": "last_name_group", "component": "Column", "children": [
    "last_name_label", "last_name_field"], "weight": 1},
  {"id": "last_name_label", "component": "Text", "text": "Last Name", "variant": "caption"},
  {"id": "last_name_field", "component": "TextField", "label": "Last Name", "value": {"path": "/}}
```
```

```

contact/lastName"},"variant":"shortText"},
{"id":"email_group","component":"Column","children":["email_label","email_field"]},
{"id":"email_label","component":"Text","text":"Email Address","variant":"caption"},
{"id":"email_field","component":"TextField","label":"Email","value":{"path":"/contact/
email"},"variant":"shortText","checks":[{"call":"required","args":{"value":{"path":"/contact/
email"}},"message":"Email is required."},{call":"email","args":{"value":{"path":"/contact/
email"}},"message":"Please enter a valid email address."}]},
{"id":"phone_group","component":"Column","children":["phone_label","phone_field"]},
{"id":"phone_label","component":"Text","text":"Phone Number","variant":"caption"},
{"id":"phone_field","component":"TextField","label":"Phone","value":{"path":"/contact/
phone"},"variant":"shortText","checks":[{"call":"regex","args":{"value":{"path":"/contact/
phone"},"pattern":"^\\d{10}$"},"message":"Phone number must be 10 digits."}]},
{"id":"pref_group","component":"Column","children":["pref_label","pref_picker"]},
{"id":"pref_label","component":"Text","text":"Preferred Contact Method","variant":"caption"},
{"id":"pref_picker","component":"ChoicePicker","variant":"mutuallyExclusive","options":
[{"label":"Email","value":"email"}, {"label":"Phone","value":"phone"},
{"label":"SMS","value":"sms"}],"value":{"path":"/contact/preference"}},
{"id":"divider_1","component":"Divider","axis":"horizontal"},
{"id":"newsletter_checkbox","component":"CheckBox","label":"Subscribe to our
newsletter","value":{"path":"/contact/subscribe"}},
{"id":"submit_button_label","component":"Text","text":"Send Message"},
{"id":"submit_button","component":"Button","child":"submit_button_label","variant":"primary"
,"action":{"event":{"name":"submitContactForm","context":
{"formId":"contact_form_1","clientTime":{"call":"formatDate","args":{"value":
"2026-02-02T15:17:00Z","format":"E MMM d, YYYY h:mm
a"},"returnType":"string"},"isNewsletterSubscribed":{"path":"/contact/subscribe"}}}}}}}}
{"version":"v0.9.1","updateDataModel":{"surfaceId":"contact_form_1","path":"/
contact","value":
{"firstName":"John","lastName":"Doe","email":"john.doe@example.com","phone":"123456789
0","preference":["email"],"subscribe":true}}}
{"version":"v0.9.1","deleteSurface":{"surfaceId":"contact_form_1"}}
```

```

## ## Component model

A2UI's component model is designed for flexibility, separating the protocol's structure from the set of available UI components.

### ### The component object

Each object in the components array of an updateComponents message defines a single UI component. It has the following structure:

- **\*\*id\*\*** (ComponentId, required): A unique string that identifies this specific component instance. This is used for parent-child references.
- **\*\*component\*\*** (string, required): Specifies the component's type (e.g., "Text").

- **\*\*Component Properties\*\***: Other properties relevant to the specific component type (e.g., text, url, children) are included directly in the component object.

This structure is designed to be both flexible and strictly validated.

### ### The component catalog

The set of available UI components and functions is defined in a Catalog. The basic catalog is defined in ``catalogs/basic/catalog.json``. While the Basic Catalog is useful for starting out, most production applications will define their own catalog to reflect their specific design system. The server must generate messages that conform to the catalog understood by the client.

### #### Catalog Identification & Compatibility

Each catalog is identified by a `catalogId` string. The `catalogId` is a string identifier used for matching catalogs between the client and server. While it is conventional to format catalog IDs as URIs (e.g., ``https://mycompany.com/catalogs/v1``) to prevent naming collisions across organizations, a `catalogId` is not required to be a resolvable network resource.

It is up to client and server developers to agree on shared catalogs with well-known IDs in order to build systems that are compatible with each other.

### ### UI composition: the adjacency list model

The A2UI protocol defines the UI as a flat list of components. The tree structure is built implicitly using ID references. This is known as an adjacency list model.

Container components (like Row, Column, List, and Card) have properties that reference the id of their child component(s). The client is responsible for storing all components in a map (e.g., ``Map<String, Component>``) and recreating the tree structure at render time.

This model allows the server to send component definitions in any order. Rendering can begin as soon as the root component is defined, with the client filling in or updating the rest of the tree progressively as additional definitions arrive.

There must be exactly one component with the ID ``root`` in the component tree, acting as the root of the component tree. Until that component is defined, other component updates will have no visible effect, and they will be buffered until a root component is defined. Once a root component is defined, the client is responsible for rendering the tree in the best way possible based on the available data, skipping invalid references.

...

Client-Side Buffer (Map)      Server Stream

Parsed and stored       
 Parsed and stored        Rendered Widget Tree

```

Parsed and stored ───┐ ── Column
                    │ ── Text: 'Welcome'
                    └─ Button

```

```

updateComponents
components: [root, title, button]

```

```

root: {id: 'root', component: 'Column', children: ['title', 'button']}
title: {id: 'title', component: 'Text', text: 'Welcome'}
button: {id: 'button', component: 'Button', child: 'button_label'}
...

```

## ## Defining actions

Interactive components (like Button) use an action property to define what happens when the user interacts with them. Actions can either trigger an event sent to the server or execute a local client-side function.

### ### Server actions

To send an event to the server, use the event property within the action object. It requires a name and an optional context.

```

```json
{
  "component": "Button",
  "text": "Submit",
  "action": {
    "event": {
      "name": "submit_form",
      "context": {
        "itemId": "123"
      }
    }
  }
}
}
```

```

### ### Local actions

To execute a local function, use the functionCall property within the action object. This property references a standard FunctionCall object.

```

```json
{

```

```

"component": "Button",
"text": "Open Link",
"action": {
  "functionCall": {
    "call": "openUrl",
    "args": {
      "url": "${url}"
    }
  }
}
}
}
}
}

```

## ## Data model representation: binding, scope

This section describes how UI components represent and reference data from the Data Model. A2UI relies on a strictly defined relationship between the UI structure (Components) and the state (Data Model), defining the mechanics of path resolution, variable scope during iteration.

### ### Path resolution & scope

Data bindings in A2UI are defined using JSON Pointers (RFC 6901). How a pointer is resolved depends on the current Evaluation Scope.

> **\*\*NOTE:\*\*** A2UI extends JSON Pointer to support Relative Paths that do not start with a forward slash ``^``. This is a deviation from strict RFC 6901 to support template-based list rendering.

Note on progressive rendering: During the initial streaming phase, data paths may resolve to undefined if the `updateDataModel` message containing that data has not yet arrived. Renderers should handle undefined values gracefully (e.g., by treating them as empty strings or showing a loading indicator) to support progressive rendering.

### ### The root scope

By default, all components operate in the Root Scope.

Paths starting with ``^`` (e.g., ``/user/profile/name``) are Absolute Paths. They always resolve from the root of the Data Model, regardless of where the component is nested in the UI tree.

### ### Collection scopes (relative paths)

When a container component (such as Column, Row, or List) utilizes the Template feature of ChildList, it creates a new Child Scope for each item in the bound array.

- **Template definition**: When a container binds its children to a path (e.g., `path: "/users"`), the client iterates over the array found at that location.
- **Scope instantiation**: For every item in the array, the client instantiates the template component.
- **Relative resolution**: Inside these instantiated components, any path that does not start with a forward slash `/` is treated as a Relative Path.
- A relative path `firstName` inside a template iterating over `/users` resolves to `/users/0/firstName` for the first item, `/users/1/firstName` for the second, etc.
- It is an error to use a non-numeric index on a path segment that refers to an array.
- **Mixing scopes**: Components inside a Child Scope can still access the Root Scope by using an Absolute Path.

### Example: scope resolution

Data model:

```
```json
{
  "company": "Acme Corp",
  "employees": [
    {"name": "Alice", "role": "Engineer"},
    {"name": "Bob", "role": "Designer"}
  ]
}
```
```

Component definition:

```
```json
{
  "id": "employee_list",
  "component": "List",
  "children": {
    "path": "/employees",
    "componentId": "employee_card_template"
  }
},
{
  "id": "employee_card_template",
  "component": "Column",
  "children": ["name_text", "company_text"]
},
{
  "id": "name_text",
  "component": "Text",
  "text": { "path": "name" }
}
```

```
// "name" is Relative. Resolves to /employees/N/name
},
{
  "id": "company_text",
  "component": "Text",
  "text": { "path": "/company" }
  // "/company" is Absolute. Resolves to "Acme Corp" globally.
}
...
```

### ### Type conversion

When a non-string value is interpolated, the client converts it to a string:

- **\*\*Numbers/Booleans\*\***: Standard string representation.
- **\*\*null/undefined\*\***: An empty string `""`.
- **\*\*Objects/Arrays\*\***: Stringified as JSON to ensure consistency across different client implementations.

### ### Two-way binding & input components

Interactive components that accept user input (TextField, CheckBox, Slider, ChoicePicker, DateTimeInput) establish a Two-Way Binding with the Data Model.

#### #### The read/write contract

Unlike static display components (like Text), input components modify the client-side data model immediately upon user interaction.

- **\*\*Read (Model -> View)\*\***: When the component renders, it reads its value from the bound path. If the Data Model is updated via `updateDataModel`, the component re-renders to reflect the new value.
- **\*\*Write (View -> Model)\*\***: When the user interacts with the component (e.g., types a character, toggles a box), the client immediately updates the value at the bound path in the local Data Model.

#### #### Reactivity

Because the local Data Model is the single source of truth, updates from input components are reactive.

If a TextField is bound to ``/user/name``, and a separate Text label is also bound to ``/user/name``, the label must update in real-time as the user types in the text field.

#### #### Server synchronization

It is critical to note that Two-Way Binding is local to the client.

- User inputs (keystrokes, toggles) do not automatically trigger network requests to the server.
- The updated state is sent to the server only when a specific User Action is triggered (e.g., a Button click).
- When an action is dispatched, the context property of the action can reference the modified data paths to send the user's input back to the server.

#### Example: form submission pattern

1. **\*\*Bind\*\***: TextField is bound to ``/formData/email``.
2. **\*\*Interact\*\***: User types "jane@example.com". The local model at ``/formData/email`` is updated.
3. **\*\*Action\*\***: A "Submit" button has the following action definition:

```
```json
"action": {
  "event": {
    "name": "submit_form",
    "context": {
      "email": { "path": "/formData/email" }
    }
  }
}
```
```

4. **\*\*Send\*\***: When clicked, the client resolves ``/formData/email`` (getting "jane@example.com") and sends it in the action payload.

### Data model updates: synchronization and convergence

While the sections above describe how components reference data, this section defines how the Data Model itself is updated and synchronized.

To support reliable data synchronization between the Renderer and the Agent that created the surface, the A2UI protocol uses a simple synchronization mechanism controlled by the `sendDataModel` property in the `createSurface` message.

#### Server to client updates

The server sends `updateDataModel` messages to modify the client's data model. These updates follow strict upsert semantics:

- If the path exists, the value is updated.
- If the path does not exist, the value is created.



- If the value is omitted (or set to undefined), the key is removed. For arrays, the value at the index is set to undefined, preserving length.

The `updateDataModel` message replaces the value at the specified path with the new content. If path is omitted (or is ``^``), the entire data model for the surface is replaced.

**\*\*Properties:\*\***

- **\*\*surfaceId\*\*** (string, required): The ID of the surface to update.
- **\*\*path\*\*** (string, optional): A JSON Pointer to the location in the data model to update. Defaults to ``^``.
- **\*\*value\*\*** (any, optional): The new value for the specified path. If omitted, the key at path is removed.

**\*\*Examples:\*\***

Update a specific field:

```
```json
{
  "version": "v0.9.1",
  "updateDataModel": {
    "surfaceId": "surface_123",
    "path": "/user/firstName",
    "value": "Alice"
  }
}
```
```

Remove a field:

```
```json
{
  "version": "v0.9.1",
  "updateDataModel": {
    "surfaceId": "surface_123",
    "path": "/user/tempData"
  }
}
```
```

Replace the entire data model:

```
```json
{
  "version": "v0.9.1",
```

```

"updateDataModel": {
  "surfaceId": "surface_123",
  "value": {
    "user": {"firstName": "Alice", "lastName": "Smith"},
    "preferences": {"theme": "dark"}
  }
}
}
}
`

```

#### #### Client to server updates

When `sendDataModel` is set to true for a surface, the client automatically appends the entire data model of that surface to the metadata of every message (such as action or user query) sent to the server that created the surface. The data model is included using the transport's metadata facility (e.g., the metadata field in A2A or a header in HTTP). The payload follows the schema in ``client_data_model.json``.

- **\*\*Targeted Delivery\*\***: The data model is sent exclusively to the server that created the surface. Data cannot leak to other agents or servers.
- **\*\*Trigger\*\***: Data is sent only when a client-to-server message is triggered (e.g., by a user action like a button click). Passive data changes (like typing in a text field) do not trigger a network request on their own; they simply update the local state, which will be sent with the next action.
- **\*\*Payload\*\***: The data model is included in the transport metadata, tagged by its `surfaceId`.
- **\*\*Convergence\*\***: The server treats the received data model as the current state of the client at the time of the action.

#### ## Client-side logic & validation

A2UI v0.9 generalizes client-side logic into Functions. These can be used for validation, data transformation, and dynamic property binding.

#### ### Registered functions

The client supports a set of named Functions (e.g., `required`, `regex`, `email`, `add`, `concat`) which are defined in the JSON schema (e.g. ``catalogs/basic/catalog.json``) alongside the component definitions. The server references these functions by name in `FunctionCall` objects. This avoids sending executable code. The client determines each function's execution boundary at runtime by reading its configuration from the active catalog definition.

Input components (like `TextField`, `CheckBox`) can define a list of checks. Each failure produces a specific error message that can be displayed when the component is rendered. Note that for validation checks, the function must return a boolean.

```

```json

```

```

"checks": [
  {
    "call": "required",
    "args": { "value": { "path": "/formData/zip" } },
    "message": "Zip code is required"
  },
  {
    "call": "regex",
    "args": {
      "value": { "path": "/formData/zip" },
      "pattern": "^[0-9]{5}$"
    },
    "message": "Must be a 5-digit zip code"
  }
]
```

```

### ### Example: button validation

Buttons can also define checks. If any check fails, the button is automatically disabled. This allows the button's state to depend on the validity of data in the model.

```

```json
{
  "component": "Button",
  "text": "Submit",
  "checks": [
    {
      "condition": {
        "call": "and",
        "args": {
          "values": [
            {
              "call": "required",
              "args": { "value": { "path": "/formData/terms" } }
            },
            {
              "call": "or",
              "args": {
                "values": [
                  {
                    "call": "required",
                    "args": { "value": { "path": "/formData/email" } }
                  },
                  {
                    "call": "required",

```

```

        "args": {"value": {"path": "/formData/phone"}}
      }
    ]
  }
}
},
"message": "You must accept terms AND provide either email or phone"
}
]
}
...

```

## ## Basic Component Catalog

The `catalogs/basic/catalog.json` provides the baseline set of components and functions.

### ### Components

| Component     | Description                                                                                 |
|---------------|---------------------------------------------------------------------------------------------|
| Text          | Displays text. Supports simple Markdown.                                                    |
| Image         | Displays an image from a URL.                                                               |
| Icon          | Displays a system-provided icon from a predefined list.                                     |
| Video         | Displays a video from a URL.                                                                |
| AudioPlayer   | A player for audio content from a URL.                                                      |
| Row           | A horizontal layout container.                                                              |
| Column        | A vertical layout container.                                                                |
| List          | A scrollable list of components.                                                            |
| Card          | A container with card-like styling.                                                         |
| Tabs          | A set of tabs, each with a title and child component.                                       |
| Divider       | A horizontal or vertical dividing line.                                                     |
| Modal         | A dialog that appears over the main content triggered by a button in the main content.      |
| Button        | A clickable button that dispatches an action. Supports 'primary' and 'borderless' variants. |
| CheckBox      | A checkbox with a label and a boolean value.                                                |
| TextField     | A field for user text input.                                                                |
| DateTimeInput | An input for date and/or time.                                                              |
| ChoicePicker  | A component for selecting one or more options.                                              |
| Slider        | A slider for selecting a numeric value within a range.                                      |

### ### Functions

| Function | Description |
|----------|-------------|
|          |             |

- | required | Checks that the value is not null, undefined, or empty. |
- | regex | Checks that the value matches a regular expression string. |
- | length | Checks string length constraints. |
- | numeric | Checks numeric range constraints. |
- | email | Checks that the value is a valid email address. |
- | formatString | Does string interpolation of data model values and registered functions. |
- | formatNumber | Formats a number with grouping and precision. |
- | formatCurrency | Formats a number as a currency string. |
- | formatDate | Formats a date/time using a pattern. |
- | pluralize | Selects a localized string based on a numeric count. |
- | openUrl | Opens a URL in a browser. |
- | and | Logical AND operation on a list of boolean values. |
- | or | Logical OR operation on a list of boolean values. |
- | not | Logical NOT operation on a boolean value. |

### ### Theme

The basic catalog defines the following theme properties that can be set in the createSurface message:

| Property         | Type   | Description                                                                                                                                                                                                               |
|------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| primaryColor     | String | The primary brand color used for highlights throughout the UI (e.g., primary buttons, active borders). The renderer may generate variants, such as lighter shades, as needed. Format: Hexadecimal code (e.g., '#00BFFF'). |
| iconUrl          | URI    | A URL for an image (e.g., logo or avatar) that identifies the agent or tool associated with the surface.                                                                                                                  |
| agentDisplayName | String | Text to be displayed next to the surface to identify the agent or tool that created it (e.g. "Weather Bot").                                                                                                              |

### ### Identity and attribution

The iconUrl and agentDisplayName fields are used to provide attribution to the user, identifying which sub-agent or tool is responsible for a particular UI surface.

In multi-agent systems or orchestrators, the orchestrator is responsible for setting or validating these fields. This ensures that the identity displayed to the user matches the actual agent server being contacted, preventing malicious agents from impersonating trusted services. For example, an orchestrator might overwrite these fields with the verified identity of the sub-agent before forwarding the createSurface message to the client.

### ### The formatString function

The formatString function supports embedding dynamic expressions directly within string properties. This allows for mixing static text with data model values and function results.

#### #### formatString syntax

Interpolated expressions are enclosed in `\${...}`. To include a literal `\${` in a string, it must be escaped as `\${`.

#### #### formatString data model binding

Values from the data model can be interpolated using their JSON Pointer path.

- `\${/user/profile/name}`: Absolute path.
- `\${firstName}`: Relative path (resolved against the current collection scope).

**\*\*Example:\*\***

```
```json
{
  "id": "user_welcome",
  "component": "Text",
  "text": {
    "call": "formatString",
    "args": {
      "value": "Hello, ${/user/firstName}! Welcome back to ${/appName}."
    }
  }
}
```
```

#### #### formatString client-side functions

Results of client-side functions can be interpolated. Function calls are identified by the presence of parentheses `()`.

- `\${now}`: A function with no arguments.
- `\${formatDate(value:\${/currentDate}, format:'yyyy-MM-dd')}`: A function with named arguments.
- Arguments can be Literals (quoted strings, numbers, or booleans), or Nested Expressions.

#### #### formatString nested interpolation

Expressions can be nested using additional `\${...}` wrappers inside an outer expression to make bindings explicit or to chain function calls.

- **\*\*Explicit Binding\*\***: `\${formatDate(value:\${/currentDate}, format:'yyyy-MM-dd')}`
- **\*\*Nested Functions\*\***: `\${upper(`\${now}`)}`

#### #### formatString type conversion

When a non-string value is interpolated, the client converts it to a string:

- **Numbers/Booleans**: Standard string representation.
- **Null/Undefined**: An empty string `""`.
- **Objects/Arrays**: Stringified as JSON to ensure consistency across different client implementations.

**## Usage pattern: the prompt-generate-validate loop**

The A2UI protocol is designed to be used in a three-step loop with a Large Language Model:

1. **Prompt**: Construct a prompt for the LLM that includes:
  - The desired UI to be generated.
  - The A2UI JSON schema, including the component catalog.
  - Examples of valid A2UI JSON.
2. **Generate**: Send the prompt to the LLM and receive the generated JSON output.
3. **Validate**: Validate the generated JSON against the A2UI schema. If the JSON is valid, it can be sent to the client for rendering. If it is invalid, the errors can be reported back to the LLM in a subsequent prompt, allowing it to self-correct.

This loop allows for a high degree of flexibility and robustness, as the system can leverage the generative capabilities of the LLM while still enforcing the structural integrity of the UI protocol.

**## Standard validation error format**

If validation fails, the client (or the system acting on behalf of the client) should send an error message back to the LLM. To ensure the LLM can understand and correct the error, use the following standard format within the error message payload:

- **code** (string, required): Must be `"VALIDATION_FAILED"`.
- **surfaceId** (string, required): The ID of the surface where the error occurred.
- **path** (string, required): The JSON pointer to the field that failed validation (e.g. `/components/0/text`).
- **message** (string, required): A short one-sentence description of why validation failed.

**Example error message:**

```
```json
{
  "error": {
    "code": "VALIDATION_FAILED",
    "surfaceId": "user_profile_card",
    "path": "/components/0/text",
    "message": "Expected stringOrPath, got integer"
  }
}
```

```
}  
}  
...
```

## ## Client-to-server messages

The protocol also defines messages that the client can send to the server, which are defined in the ``client_to_server.json`` schema. These are used for handling user interactions and reporting client-side information.

### ### action

This message is sent when the user interacts with a component that has an action defined, such as a Button.

#### **\*\*Properties:\*\***

- **\*\*name\*\*** (string, required): The name of the action.
- **\*\*surfaceId\*\*** (string, required): The ID of the surface where the action originated.
- **\*\*sourceComponentId\*\*** (string, required): The ID of the component that triggered the action.
- **\*\*timestamp\*\*** (string, required): An ISO 8601 timestamp.
- **\*\*context\*\*** (object, required): A JSON object containing any context provided in the component's action property.

## ## Capabilities & metadata

In A2UI v0.9, capabilities and other metadata are exchanged via Transport metadata or initialization payloads (e.g., A2A metadata, Agent Cards, or MCP initialization), rather than as first-class A2UI messages.

### ### Server capabilities

A server (or agent) advertises its capabilities using the ``server_capabilities.json`` schema. This indicates which catalogs it can generate UI for, and whether it accepts inline catalogs from the client. The exact mechanism depends on the transport (e.g., the `params` object in an A2A AgentCard, or server capabilities in MCP).

### ### Client capabilities

The `a2uiClientCapabilities` object in the A2A Message's metadata field follows the ``client_capabilities.json`` schema.

#### **\*\*Properties:\*\***

- **\*\*supportedCatalogIds\*\*** (array of strings, required): String identifiers of supported catalogs.



- **inlineCatalogs**: An array of inline catalog definitions provided directly by the client (useful for custom or ad-hoc components and functions).

### Client data model

When `sendDataModel` is enabled for a surface, the client includes the `a2uiClientDataModel` object in the metadata, following the ``client_data_model.json`` schema.

#### **Properties:**

- **surfaces** (object, required): A map of surface IDs to their current data models.

### error

This message is used to report a client-side error to the server.

---

---

## # §2 — A2UI Basic Catalog Implementation Guide v0.9.1

> Upstream source: ``specification/v0_9_1/docs/basic_catalog_implementation_guide.md`` (living document).

This guide is designed for renderer and client developers implementing the A2UI Basic Catalog (v0.9). It details how to visually present and functionally implement each component and client-side function defined in the catalog.

When building your framework-specific adapters (Layer 3) over the generic A2UI bindings, refer to this document for the expected visual behaviors, suggested layouts, and interaction patterns. This guide uses generic terminology applicable to Web, Mobile (iOS/Android), and Desktop platforms.

## ## 1. Components

### Text

Displays text content.

**Rendering Guidelines:** Text should be rendered using a Markdown parser when possible. If markdown rendering is unavailable or fails, gracefully fallback to rendering the raw text. In such cases, renderers should ideally attempt to strip common Markdown markers (like ``**`` or ``#``) to ensure the text remains legible and aesthetically consistent with the intended presentation.

#### **Property Mapping:**

- ``variant="h1"` through ``h5``: Apply heading styling. Suggested relative font sizes: h1 (2.5x base), h2 (2x base), h3 (1.75x base), h4 (1.5x base), h5 (1.25x base).
- ``variant="caption"`: Render as smaller text, typically italicized or in a lighter/muted color. Suggested font size: 0.8x base.
- ``variant="body"` (default): Standard body text. Uses the base font size (e.g., 16dp/16px).

### ### Image

Displays an image from a URL.

**\*\*Rendering Guidelines:\*\*** Ensure the component defaults to a flexible width so it fills its container.

**\*\*Property Mapping:\*\***

- **\*\*fit\*\***: Map the property to the platform's equivalent content scaling mode (e.g., CSS object-fit, iOS.contentMode, Android.ScaleType).
- ``variant="icon"`: Render very small and square (e.g., 24x24dp).
- ``variant="avatar"`: Render small and rounded/circular (e.g., 40x40dp, fully rounded corners).
- ``variant="smallFeature"`: Render as a small rectangle (e.g., 100x100dp).
- ``variant="mediumFeature"` (default): Render as a medium rectangle (e.g., 100% width up to 300dp, or 200x200dp).
- ``variant="largeFeature"`: Render as a large prominent image (e.g., 100% width, max height 400dp).
- ``variant="header"`: Render as a full-width banner image, usually at the top of a surface (e.g., 100% width, height 200dp, scaling mode set to cover/crop).

### ### Icon

Displays a standard system icon.

**\*\*Rendering Guidelines:\*\*** Map the icon name to a system or bundled icon set (e.g., Material Symbols, SF Symbols). The string name from the data model (e.g., accountCircle) should be converted to the required format (like snake\_case account\_circle) if required by the icon engine. Suggested styling: 24dp size and inherit the current text color.

### ### Video

A video player.

**\*\*Rendering Guidelines:\*\*** Render using a native video player component with user controls enabled. Ensure the video container spans the full width of the parent's container for responsiveness. Scrubbing (seeking) should be supported if provided by the native control.

### ### AudioPlayer

An audio player.

**\*\*Rendering Guidelines:\*\*** Render using a native audio player component with user controls enabled. Like video, its container should span the full width of its parent. Scrubbing (seeking) should be supported if provided by the native control.

### ### Row

A horizontal layout container.

**\*\*Rendering Guidelines:\*\*** Implemented using a horizontal layout container (e.g., CSS Flexbox row, Compose Row, SwiftUI HStack). Ensure it fills the available width.

**\*\*Property Mapping:\*\***

- **\*\*justify\*\***: Maps to main-axis alignment (e.g., justify-content in CSS, horizontalArrangement in Compose). Use equivalents for pushing items to edges (spaceBetween) or packing them together (start, center, end).
- **\*\*align\*\***: Maps to cross-axis alignment (e.g., align-items in CSS, verticalAlignment in Compose). Use equivalents for top (start), center, or bottom (end).

### ### Column

A vertical layout container.

**\*\*Rendering Guidelines:\*\*** Implemented using a vertical layout container (e.g., CSS Flexbox column, Compose Column, SwiftUI VStack).

**\*\*Property Mapping:\*\***

- **\*\*justify\*\***: Maps to main-axis alignment on the vertical axis.
- **\*\*align\*\***: Maps to cross-axis alignment on the horizontal axis.

### ### List

A scrollable list of components.

**\*\*Rendering Guidelines:\*\*** Children of a horizontal list should typically have a constrained max-width so they do not stretch indefinitely.

**\*\*Property Mapping:\*\***

- ``direction="vertical"`` (default): Implement as a vertically scrollable view (e.g., CSS overflow-y: auto, Compose LazyColumn, SwiftUI ScrollView vertical).
- ``direction="horizontal"``: Implement as a horizontally scrollable view. Hide the scrollbar for a cleaner look if supported by the platform.

### ### Card

A container with card-like styling that visually groups its child.

**\*\*Rendering Guidelines:\*\*** Applies a background color distinct from the main surface, rounded corners (e.g., 8dp or 12dp), a subtle shadow or elevation, and inner padding (e.g., 16dp). Note that the card accepts exactly one child. If the user wants multiple elements inside a card, they must provide a container (like Column) as the single child.

### ### Tabs

A set of tabs, each with a title and a corresponding child component.

**\*\*Rendering Guidelines:\*\*** Render a horizontal row of interactive tab headers for the titles. Visually indicate the active tab (e.g., bold text, colored bottom border).

**\*\*Behavior & State:\*\*** Maintain a local selectedIndex state (defaulting to 0). When a tab header is tapped, update selectedIndex and render only the child component that corresponds to that index.

### ### Divider

A dividing line to separate content.

**\*\*Property Mapping:\*\***

- `axis="horizontal"` (default): Render a 1dp tall line spanning 100% width with a subtle border/outline color.
- `axis="vertical"`: Render a 1dp wide line with a set height, spanning the height of the container.

### ### Modal

A dialog window.

**\*\*Rendering Guidelines:\*\***

- **\*\*Desktop UIs\*\***: Render as a centered popup or native dialog window over the main content, typically with a dimmed backdrop.
- **\*\*Mobile UIs\*\***: Render as a bottom sheet or full-screen dialog over the main content.
- You must provide a mechanism to close the modal (e.g., an "X" button, clicking/tapping the backdrop overlay, or a swipe-to-dismiss gesture).

**\*\*Behavior & State:\*\*** This component behaves differently than a standard container. It acts as a Modal Entry Point. When instantiated, the user only sees the trigger child component on the screen (which usually acts and looks like a Button). The modal logic intercepts interactions (taps/

clicks) on the trigger. When the trigger is tapped, the modal opens and displays the content child component.

### ### Button

An interactive button that dispatches a protocol action.

**\*\*Rendering Guidelines:\*\*** Render as a native interactive button component. It must render its child component inside the button (usually a Text or Icon).

**\*\*Behavior & State:\*\*** When tapped, it dispatches the action back to the server, dynamically resolving the context variables at the moment of the interaction.

**\*\*Property Mapping:\*\***

- `variant="default"`: Standard button with a subtle background and border.
- `variant="primary"`: Prominent call-to-action button using the theme's primaryColor for its background, and contrasting text.
- `variant="borderless"`: Button with no background or border, appearing like a clickable text link.

### ### TextField

A field for user text input.

**\*\*Rendering Guidelines:\*\*** Render using the platform's native text input control.

**\*\*Behavior & State:\*\*** Establishes Two-Way Binding. As the user types, immediately write the new string back to the local data model path bound to value.

**\*\*Property Mapping:\*\***

- `variant="shortText"` (default): Standard single-line input field.
- `variant="longText"`: Render as a multi-line text area.
- `variant="number"`: Render as a numeric input field, typically showing a numeric keyboard on mobile.
- `variant="obscured"`: Render as an obscured password/secure field.

### ### CheckBox

A toggleable control with a label.

**\*\*Rendering Guidelines:\*\*** Render a native checkbox or toggle switch component alongside a text label.

**\*\*Behavior & State:\*\*** Triggers two-way binding on the value path, setting it to boolean true or false when interacted with.

### ### ChoicePicker

A component for selecting one or more options from a list.

**\*\*Rendering Guidelines:\*\***

- ``displayStyle="checkbox"``` (default): Render as a dropdown menu, picker wheel, or an expanding vertical list of selectable options. A dropdown wrapper is preferred to save space.
- ``displayStyle="chips"```: Render as a horizontal, wrapping row of selectable chips/pills. Selected chips should have a distinct background/border.
- If `filterable` is true, render a text input above the list of options. As the user types, filter the visible options using a case-insensitive substring match on the option labels.

**\*\*Behavior & State:\*\*** Binds to an array of strings in the data model representing the active selections. Toggle selections in the data model upon user interaction.

### ### Slider

A control for selecting a numeric value within a range.

**\*\*Rendering Guidelines:\*\*** Render using the platform's native slider or seek bar component. Optionally display the current numeric value next to the slider track.

**\*\*Behavior & State:\*\*** Set min and max limits. Perform two-way binding, updating the numeric value path as the user drags the slider. Note that the value is a number rather than an integer, allowing for decimal ranges (e.g., 0.0 to 1.0).

### ### DateTimeInput

An input for date and/or time.

**\*\*Rendering Guidelines:\*\*** Render using native date and time picker controls.

- If `enableDate` and `enableTime` are both true, show both date and time selection UI.
- If only `enableDate` is true, show only a date picker.
- If only `enableTime` is true, show only a time picker.

**\*\*Behavior & State:\*\*** The component must convert the platform's native date/time format into a standard ISO 8601 string before writing it to the A2UI data model, and correctly parse ISO 8601 strings coming from the model into the input field.

## ## 2. Client-Side Functions

Functions provide client-side logic for validation, interpolation, and operations. As defined in the Architecture Guide, the reactivity of function arguments is generally handled by the Core Data Layer (specifically the Binder/Context layer).

Core libraries for each language (such as `@ui/web_core` for TypeScript) typically provide a complete, framework-agnostic implementation of all the functions in the basic catalog. Developers are encouraged to utilize these shared implementations rather than writing their own. When a function is called, the system resolves its arguments. If an argument is a static value, it is passed directly. If it is a dynamic binding, the Context layer handles the subscription. For most standard functions, the execute implementation simply receives a dictionary of static args and returns a static value. The Context layer wraps this execution in a reactive stream (e.g., a computed signal) so that the function re-runs whenever any of its dynamic arguments change.

However, complex functions like `formatString` must manually interact with the Context to parse and subscribe to nested dynamic dependencies.

### ### `formatString`

**\*\*Description:\*\*** The core interpolation engine. Parses the `args.value` string for ``${expression}`` blocks, combining literal strings, data paths, and other client-side function results.

**\*\*Architecture & Logic:\*\*** Because `formatString` contains dynamic expressions embedded within a string literal, the Context layer cannot pre-resolve them. The implementation must parse the string and manually create a reactive output.

- **\*\*Parser/Scanner:\*\*** Implement a parser that scans the input string (`args.value`) for ``${...}`` blocks. It must properly handle escaped markers (``\${``) which resolve to a literal ``${``.
- **\*\*Expression Evaluation:\*\*** Inside the interpolation block, the parser must differentiate between:
  - **\*\*Literals:\*\*** Quoted strings (`'...'` or `"..."`), numbers, and keywords (`true`, `false`, `null`).
  - **\*\*Data Paths:\*\*** Identifiers starting with a slash (`/absolute/path`) or relative identifiers (`relative/path`).
  - **\*\*Function Calls:\*\*** Identifiers followed by parentheses, e.g., `funcName(argName: value)`.
- **\*\*Context Resolution:\*\*** For every parsed `DataPath` or `FunctionCall` token, use the `DataContext` (e.g., `context.resolveSignal(token)`) to turn it into a reactive stream/signal.
- **\*\*Reactive Return:\*\*** The function **MUST** return a computed reactive stream (e.g., a `computed(() => ...)` signal). Inside this computed stream, unwrap all the resolved signals, convert them to strings, and concatenate them with the literal string parts.

### ### `required`

**\*\*Description:\*\*** Validates that a given value is present.

**\*\*Logic:\*\*** Return `true` if `args.value` is strictly not null, not undefined, not an empty string `""`, and not an empty array `[]`. Otherwise, return `false`.

### ### regex

**\*\*Description:\*\*** Validates a value against a regular expression.

**\*\*Logic:\*\*** Instantiate a regular expression using `args.pattern`. Test the `args.value` string against it. Return true if it matches, false otherwise.

### ### length

**\*\*Description:\*\*** Validates string length constraints.

**\*\*Logic:\*\*** Ensure the length of the string `args.value` is  $\geq$  `args.min` (if min is provided) and  $\leq$  `args.max` (if max is provided).

### ### numeric

**\*\*Description:\*\*** Validates numeric range constraints.

**\*\*Logic:\*\*** Parse `args.value` as a number. Ensure it is  $\geq$  `args.min` (if min is provided) and  $\leq$  `args.max` (if max is provided). Return true if valid, false if invalid or if it cannot be parsed as a number.

### ### email

**\*\*Description:\*\*** Validates an email address.

**\*\*Logic:\*\*** Test `args.value` against a standard email regex pattern (e.g.,  `/^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/` ).

### ### formatNumber

**\*\*Description:\*\*** Formats a numeric value.

**\*\*Logic:\*\*** Use the platform's native locale formatting (e.g., `Intl.NumberFormat` on the web or `NumberFormatter` natively) on `args.value`.

- If `args.decimals` is provided, force both the minimum and maximum fraction digits to that value.
- Enable grouping (e.g., thousands separators) unless `args.grouping` is explicitly set to false.

### ### formatCurrency

**\*\*Description:\*\*** Formats a number as a currency string.

**\*\*Logic:\*\*** Similar to `formatNumber`, but configured for currency style formatting. Apply the ISO 4217 currency code provided in `args.currency` (e.g., 'USD', 'EUR').



### ### formatDate

**\*\*Description:\*\*** Formats a timestamp into a date string.

**\*\*Logic:\*\*** Parse args.value into a native Date/Time object. Interpret the Unicode TR35 args.format string (e.g., yyyy-MM-dd, HH:mm) and construct the formatted date string. You will likely need a platform-specific date formatting library to parse the TR35 pattern.

### ### pluralize

**\*\*Description:\*\*** Returns a localized pluralized string.

**\*\*Logic:\*\*** Resolve the plural category for the numeric args.value based on the current locale (e.g., using Intl.PluralRules on the web). Map the resulting category (zero, one, two, few, many, other) to the corresponding string provided in the args object. If the specific category string is missing from args, fallback to args.other.

### ### openUrl

**\*\*Description:\*\*** Opens a URL.

**\*\*Logic:\*\*** Open args.url using the native platform's URL handler (e.g., opening in the system browser or deep-linking to an app). This function returns void and is executed as a side-effect.

**\*\*Security Constraints & Implementation Requirements (Mandatory):\*\*** To prevent DOM-Based Cross-Site Scripting (XSS) via `javascript:`, `data:`, or other non-HTTP schemes:

1. **\*\*Resolve Relative URLs\*\*:** Before validation, resolve relative paths against the current environment context (e.g., window.location.href in browsers) using standard URL parsing.
2. **\*\*Enforce Scheme Allowlist\*\*:** Strictly validate that the resolved URL protocol/scheme is either `https:` or `http:`. Throw an execution or runtime error (such as A2uiExpressionError) and abort the action if any other scheme is used.
3. **\*\*Tab-Nabbing Protection\*\*:** When opening URLs in a new browser window/tab, always supply security attributes: `noopener,noreferrer` (e.g. `window.open(url, '\_blank', 'noopener,noreferrer')`).

### ### and

**\*\*Description:\*\*** Logical AND operator.

**\*\*Logic:\*\*** Iterate through the boolean array args.values. Return true only if all values are true. Short-circuit evaluation is encouraged.

### ### or

**\*\*Description:\*\*** Logical OR operator.

**\*\*Logic:\*\*** Iterate through the boolean array `args.values`. Return true if at least one value is true. Short-circuit evaluation is encouraged.

### not

**\*\*Description:\*\*** Logical NOT operator.

**\*\*Logic:\*\*** Return the strict boolean negation of `args.value`.

### ## 3. Layout Spacing: Margins and Padding

A common challenge in dynamic UI frameworks is preventing "spacing multiplication," where nested containers (e.g., a Text inside a Row inside a Column) result in accumulated empty space that throws off the design.

To achieve a clean, consistent default spacing where elements feel naturally separated without stacking empty space, implementers should follow a **\*\*Leaf-Margin Strategy\*\***:

- **\*\*Invisible Containers have ZERO Spacing\*\***: Structural, invisible layout containers (Row, Column, List) should have no internal padding and no external margins. They act purely as structural boundaries. This guarantees that wrapping an element in a Row or Column does not alter its spacing.
- **\*\*Leaf Components carry the Margin\*\***: All non-container, visual "leaf" elements (Text, Image, Icon, Video, AudioPlayer, Slider, etc.) should have a uniform default external margin applied to them (e.g., 8dp on all sides).
- **\*\*Visually Outlined Containers carry the Margin\*\***: Containers and inputs that have a visible boundary (Card, Button, TextField, CheckBox, ChoicePicker) should also apply this same uniform default external margin.
  - Note: These elements will naturally also need internal padding to keep their content away from their own visible borders, but this padding is localized and does not affect the external layout.

**\*\*Why use Margins on Leaves?\*\*** Applying margins directly to the visual elements—rather than relying on padding or gap properties on the parent containers—ensures predictable spacing. For example, if you have `Row(Item1, Item2)`, using margins on the items guarantees that there is space to the left of Item1, space to the right of Item2, and space between them. Because the invisible containers themselves contribute zero extra spacing, you can deeply nest your structural rows and columns without the spacing unexpectedly multiplying.

### ## 4. Color, Contrast, and Nesting

A common challenge in dynamically generated UI is ensuring proper contrast and visual hierarchy when components are nested. For example:

- A Text or Icon nested inside a primary Button must change its color to contrast with the button's background.
- A Card nested inside another Card should remain visually distinct.

To keep the A2UI rendering layer simple and performant, do not manually calculate or pass color properties down the A2UI component tree. Instead, rely entirely on the native context and theme inheritance mechanisms provided by your target UI framework.

### ### Text and Icon Contrast

When an element defines a strong background color (like a primary Button using the theme's `primaryColor`), it must also define the expected text color for its content. It should propagate this expectation implicitly.

- **Web (CSS)**: The Button wrapper sets the standard CSS `color` property. Because color is inherited in CSS, any Text or Icon component rendered inside the button will automatically adapt.
- **Compose (Android)**: The button wrapper should use `CompositionLocalProvider(LocalContentColor provides ...)`. Any nested Text and Icon components will automatically pick up this color without needing it explicitly passed to their A2UI classes.
- **SwiftUI (iOS)**: Apply `.foregroundColor(...)` or `.environment(\.colorScheme, ...)` to the button wrapper.
- **Flutter**: Use `DefaultTextStyle.merge()` and `IconTheme.merge()` within the button wrapper. If using standard Material buttons (like `ElevatedButton`), this is often handled for you automatically.

**Rule of Thumb:** Leaf components like Text and Icon should never hardcode their colors unless explicitly instructed by a property. They must always inherit from their environment.

### ### Nesting Containers (Cards)

When a Card is nested within another Card, or placed on different background surfaces, it needs to remain distinct. Attempting to alternate surface colors based on depth adds significant complexity to the renderer.

**Recommended Approach: Outlines and Transparent Surfaces** — The simplest, most robust starting approach is to give Card components a transparent background and a visible outline/border (e.g., a 1dp outline matching the theme's outline/border color).

- By using borders instead of opaque surface colors, nested cards will simply draw an inner boundary within the parent card.
- This guarantees a clear visual hierarchy regardless of how deeply they are nested, and it requires zero context-passing or depth-tracking in your code.

- If your design system requires opaque cards, consider using a framework-specific elevation system (e.g., standard Material elevation) which often handles shadow and surface tinting automatically, rather than building custom color-alternation logic into the A2UI adapters.

---  
---

### # §3 — A2UI Extension for A2A Protocol v0.9.1

> Upstream source: ``specification/v0_9_1/docs/a2ui_extension_specification.md`` (living document).

> **\*\*Version Compatibility:\*\*** This extension specification applies to A2UI v0.9.1 and the A2A Protocol. For the base A2UI protocol, see §1 (v0.9.1 Protocol Specification).

#### ## Overview

This document is intended for developers implementing the A2UI A2A extension. The extension adds A2UI v0.9.1 support to A2A, a format for agents to send streaming, interactive user interfaces to clients.

Note that A2UI extension activation is optional as clients and agents can negotiate A2UI support using ``message.metadata["a2uiClientCapabilities"]`` which is attached to every A2A message from the client and contains the supported protocol version and catalogs. Agents advertising A2UI support in their AgentCard is encouraged as clients may rely on it to determine if they should send ``message.metadata["a2uiClientCapabilities"]``, however it is not explicitly required.

#### ## Extension URI

The URI of this extension is:

---

<https://a2ui.org/a2a-extension/a2ui/v0.9.1>

---

This URI is the canonical way to communicate protocol versioning between clients and agents. The extension URI explicitly encodes the version (e.g., v0.9.1). A client requesting this specific URI indicates it supports the v0.9.1 schema format.

#### ## Agent Card

Agents are encouraged to advertise their A2UI capabilities in their AgentCard within the ``AgentCapabilities.extensions`` list. This advertisement is optional, but it informs the client whether to send ``message.metadata["a2uiClientCapabilities"]``. The params object defines the agent's specific UI support and corresponds directly to the Server Capabilities Schema.

**\*\*Example AgentCard payload:\*\***

```

```json
{
  "name": "Dashboard Agent",
  "description": "Agent capable of generating dynamic UI dashboards.",
  "capabilities": {
    "extensions": [
      {
        "uri": "https://a2ui.org/a2a-extension/a2ui/v0.9.1",
        "description": "Ability to render A2UI v0.9.1",
        "required": false,
        "params": {
          "supportedCatalogIds": [
            "https://a2ui.org/specification/v0_9/catalogs/basic/catalog.json",
            "https://my-company.com/a2ui/v0.9/my_custom_catalog.json"
          ],
          "acceptsInlineCatalogs": true
        }
      }
    ]
  }
}
```

```

The params object corresponds to the `v0.9.1` object in the `server\_capabilities.json` schema:

- **params.supportedCatalogIds** (optional): An array of strings, where each string is an ID identifying a Catalog Definition Schema that the agent can generate. This is not necessarily a resolvable URI.
- **params.acceptsInlineCatalogs** (optional): A boolean indicating if the agent can accept an inlineCatalogs array in the client's a2uiClientCapabilities. If omitted, this defaults to false.

## ## A2A Extension activation

Activating the A2UI extension is optional. Clients and agents can negotiate A2UI support using `message.metadata["a2uiClientCapabilities"]` and `DataPart.data.metadata["mimeType"] = "application/a2ui+json"`.

Specifically:

- If a client includes `message.metadata["a2uiClientCapabilities"]`, the agent can use this object to determine the supported A2UI protocol version and catalogs.
- If an agent returns an A2A DataPart with `data.metadata["mimeType"] = "application/a2ui+json"`, the client knows the payload contains A2UI messages.

- While explicit activation is not required, clients can still explicitly activate the extension using the transport-defined A2A extension activation mechanism. The A2A Extensions Guide defines this process.

> **Note:** You should not use ``accepted_output_modes: ['a2ui']`` (which is not an A2UI standard) to trigger A2UI.

### ### JSON-RPC and HTTP transports

To activate the A2UI A2A Extension, the ``X-A2A-Extensions`` HTTP header includes the extension URI.

**Example HTTP SendMessageRequest:**

```
``http
POST /v1/messages HTTP/1.1
Host: agent.example.com
X-A2A-Extensions: https://a2ui.org/a2a-extension/a2ui/v0.9.1
Content-Type: application/json
```

```
{
  "message": {
    "parts": [
      {
        "text": "Hello, show me the dashboard"
      }
    ]
  }
}
```

### ### GRPC transport

To activate the A2UI A2A Extension, the client adds the extension URI to A2A ``sendMessageParams.metadata["X-A2A-Extensions"]``.

**Example gRPC SendMessageRequest:**

```
``json
{
  "metadata": {
    "X-A2A-Extensions": "https://a2ui.org/a2a-extension/a2ui/v0.9.1"
  },
  "message": {
    "parts": [
      {
```

```

    "text": "Hello, show me the dashboard"
  }
]
}
}
...

```

## ## A2A Client to Server Metadata

Clients attach `a2uiClientCapabilities` and `a2uiClientDataModel` to A2A messages to communicate their state and supported catalogs.

### ### a2uiClientCapabilities

The client sends ``sendMessageRequest.message["a2uiClientCapabilities"]`` (Client Capabilities Schema) to advertise which catalogs the renderer supports.

**\*\*Example SendMessageRequest with Capabilities:\*\***

```

```json
{
  "message": {
    "parts": [
      {
        "text": "Show me the dashboard."
      }
    ],
    "metadata": {
      "a2uiClientCapabilities": {
        "v0.9.1": {
          "supportedCatalogIds": [
            "https://a2ui.org/specification/v0_9/catalogs/basic/catalog.json",
            "https://my-company.com/a2ui/v0.9/my_custom_catalog.json"
          ]
        }
      }
    }
  }
}
}
...

```

### ### a2uiClientDataModel

When a surface enables Data Model Sync, the client sends ``sendMessageRequest.message["a2uiClientDataModel"]`` (Client Data Model Schema) on every

message. This model provides the agent with the latest UI state. For more details, see the Actions Guide.

**\*\*Example SendMessageRequest with Data Model:\*\***

```
```json
{
  "message": {
    "parts": [
      {
        "text": "Submit the form."
      }
    ],
    "metadata": {
      "a2uiClientDataModel": {
        "version": "v0.9.1",
        "surfaces": {
          "main_surface_id": {
            "user_id": "12345",
            "email": "user@example.com"
          }
        }
      }
    }
  }
}
```
```

## ## Data encoding

Agents and clients encode A2UI messages as an A2A DataPart.

To identify a DataPart as containing A2UI data, it must have the following metadata:

```
```
DataPart.data.metadata["mimeType"] = "application/a2ui+json"
```
```

The data field of the DataPart contains a list of A2UI JSON messages (e.g., createSurface, updateComponents, action). It **MUST** be an array of messages.

## ## Processing Rules

The data field contains a list of messages. This list is **NOT** a transactional unit. Receivers (both Clients and Agents) **MUST** process messages in the list sequentially.



- If a single message in the list fails to validate or apply (e.g., due to a schema violation or invalid reference), the receiver SHOULD report/log the error for that specific message and MUST continue processing the remaining messages in the list.
- Atomicity is guaranteed only at the individual message level. However, for a better user experience, a renderer SHOULD NOT repaint the UI until all messages in the list have been processed. This prevents intermediate states from flickering to the user.

## ## Server-to-client messages

When an agent sends a message to a client (or another agent acting as a client/renderer), the data payload must validate against the Server-to-Client Message List Schema.

**\*\*Example DataPart:\*\***

```
```json
{
  "data": [
    {
      "version": "v0.9.1",
      "createSurface": {
        "surfaceId": "example_surface",
        "catalogId": "https://a2ui.org/specification/v0_9/catalogs/basic/catalog.json"
      }
    },
    {
      "version": "v0.9.1",
      "updateComponents": {
        "surfaceId": "example_surface",
        "components": [
          {
            "Text": {
              "id": "root",
              "text": "Hello!"
            }
          ]
        }
      }
    ]
  },
  "kind": "data",
  "metadata": {
    "mimeType": "application/a2ui+json"
  }
}
```
```

## ## Client-to-server events

When a client (or an agent forwarding an event) sends a message to an agent, it also uses a DataPart with the same `application/a2ui+json` MIME type. However, the data payload must validate against the Client-to-Server Message List Schema.

**\*\*Example action DataPart:\*\***

```
```json
{
  "data": [
    {
      "version": "v0.9.1",
      "action": {
        "name": "submit_form",
        "surfaceId": "contact_form_1",
        "sourceComponentId": "submit_button",
        "timestamp": "2026-01-15T12:00:00Z",
        "context": {
          "email": "user@example.com"
        }
      }
    }
  ],
  "kind": "data",
  "metadata": {
    "mimeType": "application/a2ui+json"
  }
}
```
```

---

\*End of SPECIFICATIONS.md — snapshot taken 2026-07-27 from <https://a2ui.org/>. Check upstream for updates (living documents).\*